Below is a **master table of Java class types/structures you typically implement when building algorithms in Java**, along with **use cases**, **software implementation examples**, and **real-life analogies**.

✅ Java Classes Implemented Based on Algorithms (Master Table)

Java Class Type / Pattern	What it Represents (Algorithm Perspective)	Typical Algorithms That Use It	Common Use Cases in Software	Real-life Example
Utility Class (Static Methods)	Algorithm functions only (no state)	Sorting, Searching, Math, Hashing	SortUtils.sort(), BinarySearchUtils.find()	Calculator formula book
Service Class / Business Algorithm Class	Algorithm + business rules	Recommendation, Pricing, Matching	PricingService, FraudDetectionService	Cashier applying discount rules
Model / DTO Class	Data container for algorithm input/output	ML features, graph nodes, DP states	UserDTO, TransactionDTO	Patient report sheet
Entity Class (JPA/Hibernate)	Database-backed objects for algorithms	Aggregation, analytics, selection	UserEntity, OrderEntity	Warehouse inventory register
Comparator Class	Custom ordering logic	Sorting, Ranking	SortBySalaryComparator	Ranking students by marks
Predicate / Filter Class	Rule-based filtering	Filtering algorithms	Spam filter rules, validation filters	Security guard checking ID
Strategy Class (Strategy Pattern)	Interchangeable algorithms	Sorting strategy, routing strategy	PaymentStrategy, CompressionStrategy	Choosing travel route
Factory Class	Creates correct algorithm implementation	Strategy selection	AlgorithmFactory.getAlgo(type)	Vending machine selection
Builder Class	Step-by-step construction for complex	ML pipeline configs	RequestBuilder, QueryBuilder	Building a customized burger

	algorithm inputs			
Validator Class	Input constraints checking for algorithms	Parsing, data cleaning	EmailValidator, SchemaValidator	Entry gate ticket validation
Parser Class	Converts raw data into structured format	Tokenizing, lexical parsing	JSON/XML/CSV parser	Translator converting languages
Tokenizer / Lexer Class	Splits input into tokens	NLP, compilers	Tokenizer.tokenize (text)	Breaking sentence into words
Dynamic Programming State Class	Maintains memoization/ table	DP algorithms	knapsack, LIS, edit-distance	Remembering solved puzzles
Memoization Cache Class	Cache previously computed results	DP / recursion optimization	caching results for API calls	Saving repeated calculations
Graph Node Class	Represents graph structure	BFS, DFS, Dijkstra, MST	social networks, map routing	City connections map
Tree Node Class	Represents hierarchical structure	BST, AVL, Segment Tree, Trie	file system, search indexing	Family tree
Trie Class	Prefix search structure	Autocomplete, dictionary	Search suggestions	Phone contact search
Heap / Priority Queue Wrapper Class	Manages min/max ordering	Dijkstra, Top-K, Scheduling	task scheduling, load balancing	Hospital patient priority
Union-Find / Disjoint Set Class	Component merging tracking	Kruskal MST, clustering	network connectivity	groups merging in society
Scheduler Class	Assigns jobs using algorithms	Round robin, priority scheduling	CPU scheduling	Train timetable system

Rate Limiter Class	Controls frequency using algorithm	Token bucket/leaky bucket	API rate limiting	Gate entry limit per minute
Load Balancer Class	Distributes requests	round-robin, weighted	distributed systems	distributing customers in queues
Indexer Class	Builds search index	inverted index	search engine	library catalog
Searcher Class	Finds results from indexed data	binary search, BFS/DFS	product search	searching items in shop
Segment Tree / Fenwick Tree Class	Fast range queries	query/update algorithms	analytics counters	scoreboard updates
Hashing Class	Generates hash values	hashing, bloom filter	caching, password storage	fingerprint system
Bloom Filter Class	Probabilistic membership test	bloom filter algorithm	cache/DB pre-check	quick “maybe exists?” check
Queue Manager Class	Manages processing queue	BFS, event-driven queues	message brokers	token queue in bank
Stack Manager Class	Manages stack operations	DFS, expression evaluation	parsing, undo feature	pile of plates
Backtracking Solver Class	Explores all combinations	N-Queens, Sudoku, subsets	scheduling constraints	trying all lock combinations
Greedy Solver Class	Picks local best decision	activity selection, coin change	resource allocation	choosing cheapest best option
Divide & Conquer Solver Class	split-recursive merge	merge sort, quick sort	sorting large data	divide work among teams
Randomized Algorithm Class	probabilistic decision	randomized quicksort, Monte Carlo	security, ML sampling	lottery selection

Simulation Engine Class	imitates real system	event simulation	traffic simulation	flight simulation
ML Model Wrapper Class	encapsulates ML algorithm	regression/classification	prediction service	doctor predicting disease risk
Recommendation Engine Class	ranking + filtering	collaborative filtering	OTT movie suggestions	recommending products in shop
Fraud Detection Class	anomaly + thresholds	clustering, rules	banking fraud detection	suspicious behavior detector
Graph Path Finder Class	shortest route logic	Dijkstra, A*	map navigation	GPS route
Distributed Consensus Class	agreement algorithm	Paxos/Raft	cluster coordination	group decision making
Encryption / Cipher Class	algorithm for securing	AES/RSA	secure data transfer	lock-and-key protection
Compression Class	data shrink algorithms	Huffman/LZW	file compression	packing suitcase efficiently
Logger / Monitoring Class	collects stats using algorithms	streaming aggregation	observability	CCTV monitoring
Statistics / Metrics Class	aggregates data	mean, median, histogram	analytics dashboards	election vote counting
Stream Processor Class	real-time processing	sliding window, aggregation	kafka streaming	continuous sensor tracking
Finite State Machine (FSM) Class	state transitions	automata-based algorithm	payment workflow	traffic light system
Event Handler Class	reacts to triggers	event-driven algorithms	UI clicks, messaging	receptionist responding to calls
Rule Engine Class	applies rules systematically	decision trees / logic rules	credit checks	judge applying laws

---

✅ Conclusion (Interview-ready)

In real IT projects, **algorithms are not written as "just functions"**. They are converted into clean Java components such as:

- Solver / Engine / Service classes
- Node / State / Cache classes
- Strategy / Factory / Validator classes
  which improves:
  ✅ maintainability
  ✅ testing
  ✅ scalability
  ✅ real-world integration

If you want, I can also provide:

✅ **"Algorithm → Best Java Class Type" mapping table (100+ rows)**
✅ **Java Spring Boot directory structure showing where each algorithm class belongs**
✅ **Mini project example (Search Engine / Fraud Detector / Recommendation Engine) with real Java code**

Absolutely ✅
Below is the **MASTER "Algorithm → Best Java Class Type" mapping table (100+ rows)**.

This table tells you:

- **Which algorithm** you are implementing
- **Which Java class type fits best** (Service / Strategy / Utility / Solver / Engine / Node / Cache / Comparator etc.)
- **How it is used in real software systems**

✅ Algorithm → Best Java Class Type Mapping Table (100+ Rows)

Legend (Java Class Type Keywords)

- **Utility** = static helpers (SortUtils, MathUtils)
- **Solver** = algorithm execution (KnapsackSolver)
- **Engine** = multi-step pipeline (RecommendationEngine)
- **Service** = business + algorithm (FraudDetectionService)
- **Strategy** = interchangeable algorithms (SortingStrategy)
- **Factory** = picks algorithm implementation (AlgorithmFactory)
- **Model/DTO** = inputs/outputs (GraphRequestDTO)
- **Node/State** = structure of algo (TreeNode, DPState)

- **Cache** = memoization (MemoCache)
- **Validator** = input checks (InputValidator)
- **Comparator** = ranking/sorting (SalaryComparator)
- **Processor** = streaming/event flow (StreamProcessor)

✅ 1) Searching Algorithms (10 rows)

#	Algorithm	Best Java Class Type	Typical Java Class Name
1	Linear Search	Utility	SearchUtils
2	Binary Search	Utility	BinarySearchUtils
3	Jump Search	Utility	JumpSearchUtils
4	Interpolation Search	Utility	InterpolationSearchUtils
5	Exponential Search	Utility	ExponentialSearchUtils
6	Fibonacci Search	Utility	FibonacciSearchUtils
7	Ternary Search	Utility	TernarySearchUtils
8	BFS Search (graph)	Solver + Node	BFSSolver, GraphNode
9	DFS Search (graph)	Solver + Node	DFSSolver, GraphNode
10	A* Search	Solver + Strategy	AStarSolver, HeuristicStrategy

---

✅ 2) Sorting Algorithms (15 rows)

#	Algorithm	Best Java Class Type	Typical Java Class Name
11	Bubble Sort	Utility	SortUtils
12	Selection Sort	Utility	SelectionSortUtils
13	Insertion Sort	Utility	InsertionSortUtils
14	Merge Sort	Utility	MergeSortUtils
15	Quick Sort	Utility	QuickSortUtils
16	Heap Sort	Utility + Heap Wrapper	HeapSortUtils, Heap
17	Counting Sort	Utility	CountingSortUtils
18	Radix Sort	Utility	RadixSortUtils
19	Bucket Sort	Utility	BucketSortUtils
20	Shell Sort	Utility	ShellSortUtils
21	TimSort (Java Arrays.sort)	Strategy	TimSortStrategy
22	Top-K Sorting	Solver + Heap	TopKSolver, Heap
23	Partial Sort	Solver	PartialSortSolver
24	Stable Sort	Strategy	StableSortingStrategy
25	Custom Object Sorting	Comparator	EmployeeSalaryComparator

✅ 3) Greedy Algorithms (10 rows)

#	Algorithm	Best Java Class Type	Typical Java Class Name

26	Activity Selection	Solver	ActivitySelectionSolver
27	Fractional Knapsack	Solver	FractionalKnapsackSolver
28	Job Sequencing	Solver	JobSchedulingSolver
29	Huffman Coding	Engine + Node	HuffmanEngine, HuffmanNode
30	Prim MST (greedy)	Solver + Graph	PrimMstSolver, Graph
31	Kruskal MST	Solver + DSU	KruskalMstSolver, UnionFind
32	Dijkstra shortest path	Solver + Heap	DijkstraSolver, MinHeap
33	Coin Change (greedy version)	Solver	CoinChangeGreedySolver
34	Interval Scheduling	Solver	IntervalSchedulingSolver
35	Load Balancing Greedy	Service + Solver	LoadBalancerService, BalancerSolver

---

✅ 4) Dynamic Programming (DP) Algorithms (15 rows)

#	Algorithm	Best Java Class Type	Typical Java Class Name
36	Fibonacci DP	Utility + Cache	FibonacciUtils, MemoCache
37	0/1 Knapsack	Solver + DPState	KnapsackSolver, DPState
38	Unbounded Knapsack	Solver + DPState	UnboundedKnapsackSolver
39	Longest Common Subsequence (LCS)	Solver	LCSSolver
40	Longest Increasing Subsequence (LIS)	Solver	LISSolver
41	Edit Distance	Solver	EditDistanceSolver
42	Matrix Chain Multiplication	Solver	MatrixChainSolver
43	Coin Change DP	Solver	CoinChangeDPSolver
44	Partition Equal Subset Sum	Solver	PartitionSolver
45	Minimum Path Sum Grid	Solver	GridPathDPSolver
46	House Robber	Solver	HouseRobberSolver
47	Rod Cutting	Solver	RodCuttingSolver
48	DP on Trees	Solver + TreeNode	TreeDPSolver, TreeNode
49	DP with Bitmask	Solver	BitmaskDPSolver
50	Palindrome Partition DP	Solver	PalindromePartitionSolver

✅ 5) Backtracking Algorithms (10 rows)

#	Algorithm	Best Java Class Type	Typical Java Class Name
51	N-Queens	Solver	NQueensSolver
52	Sudoku Solver	Solver	SudokuSolver
53	Permutations	Utility/Solver	PermutationSolver
54	Subsets generation	Utility/Solver	SubsetGenerator
55	Combination Sum	Solver	CombinationSumSolver
56	Word Search	Solver	WordSearchSolver

57	Rat in a Maze	Solver	MazeSolver
58	Graph Coloring	Solver	GraphColoringSolver
59	Hamiltonian Path	Solver	HamiltonianPathSolver
60	Crossword Fill	Solver	CrosswordSolver

✅ 6) Divide & Conquer (8 rows)

#	Algorithm	Best Java Class Type	Typical Java Class Name
61	Merge Sort	Utility	MergeSortUtils
62	Quick Sort	Utility	QuickSortUtils
63	Karatsuba Multiplication	Utility/Solver	KaratsubaSolver
64	Closest Pair of Points	Solver	ClosestPairSolver
65	Strassen Matrix Multiplication	Solver	StrassenMatrixSolver
66	Binary Search on Answer	Solver	BinarySearchAnswerSolver
67	Count Inversions	Solver	InversionCountSolver
68	Divide&Conquer Max Subarray	Solver	MaxSubarraySolver

✅ 7) Graph Algorithms (15 rows)

#	Algorithm	Best Java Class Type	Typical Java Class Name
69	BFS	Solver + GraphNode	BFSSolver, GraphNode
70	DFS	Solver + GraphNode	DFSSolver, GraphNode
71	Dijkstra	Solver + Heap	DijkstraSolver, MinHeap
72	Bellman-Ford	Solver	BellmanFordSolver
73	Floyd Warshall	Solver	FloydWarshallSolver
74	Topological Sort	Solver	TopoSortSolver
75	Detect Cycle (directed)	Solver	DirectedCycleDetector
76	Detect Cycle (undirected)	Solver	UndirectedCycleDetector
77	Strongly Connected Components	Solver	TarjanSccSolver
78	Kosaraju SCC	Solver	KosarajuSccSolver
79	Minimum Spanning Tree	Solver + DSU	MstSolver, UnionFind
80	Bridges in Graph	Solver	BridgeFinderSolver
81	Articulation Points	Solver	ArticulationPointSolver
82	Bipartite Check	Solver	BipartiteChecker
83	Max Flow (Ford-Fulkerson)	Solver + Graph	MaxFlowSolver, ResidualGraph

---

✅ 8) Tree Algorithms (10 rows)

#	Algorithm	Best Java Class Type	Typical Java Class Name
84	BST Insert/Search/Delete	Node + Service	BSTNode, BSTService
85	Tree Traversal (In/Pre/Post)	Utility	TreeTraversalUtils
86	Level Order Traversal	Utility/Solver	LevelOrderTraversalSolver

87	Lowest Common Ancestor (LCA)	Solver	LCASolver
88	Binary Tree Diameter	Solver	TreeDiameterSolver
89	Segment Tree	DataStructure Class	SegmentTree
90	Fenwick Tree (BIT)	DataStructure Class	FenwickTree
91	Trie Insert/Search	DataStructure Class	Trie
92	AVL Tree balancing	DataStructure Class	AVLTree
93	Red Black Tree	DataStructure Class	RedBlackTree

✅ 9) String / Pattern Matching (12 rows)

#	Algorithm	Best Java Class Type	Typical Java Class Name
94	Naive Pattern Search	Utility	PatternSearchUtils
95	KMP Algorithm	Solver	KMPSolver
96	Rabin-Karp	Solver + Hashing	RabinKarpSolver, RollingHash
97	Z Algorithm	Solver	ZAlgorithmSolver
98	Boyer-Moore	Solver	BoyerMooreSolver
99	Trie-based matching	Trie Class	TrieMatcher
100	Regex Matching Engine	Engine/FSM	RegexEngine
101	Palindrome Check	Utility	PalindromeUtils
102	Longest Palindromic Substring	Solver	PalindromeSubstringSolver
103	Suffix Array	DataStructure Class	SuffixArray
104	Suffix Tree	DataStructure Class	SuffixTree
105	LCP Array	Utility	LCPUtils

✅ 10) Hashing / Probabilistic Algorithms (10 rows)

#	Algorithm	Best Java Class Type	Typical Java Class Name
106	Hash Table	DataStructure Class	MyHashMap
107	Rolling Hash	Utility	RollingHash
108	Bloom Filter	DataStructure Class	BloomFilter
109	Count-Min Sketch	DataStructure Class	CountMinSketch
110	HyperLogLog	DataStructure Class	HyperLogLogCounter
111	Reservoir Sampling	Solver	ReservoirSampler
112	Monte Carlo Simulation	Simulation Engine	MonteCarloEngine
113	Las Vegas Algorithm	Solver	LasVegasSolver
114	Locality Sensitive Hashing	Engine	LSHEngine
115	MinHash	Utility/Engine	MinHashEngine

✅ 11) Encryption / Compression Algorithms (8 rows)

#	Algorithm	Best Java Class Type	Typical Java Class Name
116	AES Encryption	Service	EncryptionService
117	RSA Encryption	Service	RsaCipherService

118	SHA Hashing	Utility	HashUtils
119	Base64 Encoding	Utility	EncodingUtils
120	Huffman Compression	Engine + Node	HuffmanCompressionEngine, HuffmanNode
121	LZ Compression	Engine	LZCompressionEngine
122	GZIP Compression	Service	CompressionService
123	CRC Checksum	Utility	ChecksumUtils

✅ 12) Scheduling / OS Algorithms (10 rows)

#	Algorithm	Best Java Class Type	Typical Java Class Name
124	FCFS Scheduling	Scheduler	FcfsScheduler
125	SJF Scheduling	Scheduler	SjfScheduler
126	Priority Scheduling	Scheduler	PriorityScheduler
127	Round Robin	Scheduler	RoundRobinScheduler
128	Deadlock Detection	Solver	DeadlockDetector
129	Banker's Algorithm	Solver	BankersAlgorithmSolver
130	Page Replacement (LRU)	Cache Class	LRUCache
131	Page Replacement (FIFO)	Cache Class	FIFOCache
132	Rate Limiting (Token Bucket)	Service	TokenBucketRateLimiter
133	Load Balancing	Service	LoadBalancerService

✅ Total Rows: 133 ✅

So yes — this is a **100+ rows complete mapping** and interview-ready.

If you want next, I can generate:

✅ **"Algorithm → Big-O → Recommended Java Data Structure → Class Type" mega table**
✅ A full **Spring Boot project template** with these class names under packages like:
controller/ service/ engine/ solver/ strategy/ model/ ds/ utils/

Absolutely ✅
Below is a **perfect industry-style Spring Boot directory structure** showing **exactly where each algorithm-related class belongs** (Utility, Solver, Engine, Strategy, Factory, Cache, Validator, Comparator, Node/State, etc.).

This is the **most important thing** in real software: algorithms should NOT be written randomly in one package.

✅ Java Spring Boot Directory Structure (Algorithm Classes Placement)

✅ Recommended Base Package

Example project name:

algorithm-hub-springboot

Base package:

com.company.algorithmhub

✅ 1) FULL Spring Boot Project Structure (Best Practice)

```
algorithm-hub-springboot/
├── pom.xml
├── README.md
└── src/
    ├── main/
    │   ├── java/
    │   │   └── com/company/algorithmhub/
    │   │       ├── AlgorithmHubApplication.java
    │   │       │
    │   │       ├── config/
    │   │       │   ├── SwaggerConfig.java
    │   │       │   ├── SecurityConfig.java
    │   │       │   └── CacheConfig.java
    │   │       │
    │   │       ├── controller/
    │   │       │   ├── SearchController.java
    │   │       │   ├── SortController.java
    │   │       │   ├── GraphController.java
    │   │       │   ├── DPController.java
    │   │       │   ├── BacktrackingController.java
    │   │       │   ├── StringAlgoController.java
    │   │       │   └── CryptoCompressionController.java
    │   │       │
    │   │       ├── dto/
    │   │       │   ├── request/
    │   │       │   │   ├── SearchRequest.java
    │   │       │   │   ├── SortRequest.java
    │   │       │   │   ├── GraphRequest.java
    │   │       │   │   ├── KnapsackRequest.java
    │   │       │   │   └── CompressionRequest.java
    │   │       │   │
    │   │       │   └── response/
    │   │       │       ├── SearchResponse.java
    │   │       │       ├── SortResponse.java
```

```
│   │   │   ├── GraphPathResponse.java
│   │   │   ├── KnapsackResponse.java
│   │   │   └── CompressionResponse.java
│   │   │
│   │   ├── service/
│   │   │   ├── AlgorithmService.java
│   │   │   ├── SearchService.java
│   │   │   ├── SortService.java
│   │   │   ├── GraphService.java
│   │   │   ├── DpService.java
│   │   │   ├── BacktrackingService.java
│   │   │   ├── CryptoService.java
│   │   │   ├── CompressionService.java
│   │   │   └── RecommendationService.java
│   │   │
│   │   ├── engine/
│   │   │   ├── RecommendationEngine.java
│   │   │   ├── FraudDetectionEngine.java
│   │   │   ├── SearchEngine.java
│   │   │   ├── IndexingEngine.java
│   │   │   └── StreamProcessingEngine.java
│   │   │
│   │   ├── solver/
│   │   │   ├── search/
│   │   │   │   ├── BinarySearchSolver.java
│   │   │   │   ├── LinearSearchSolver.java
│   │   │   │   ├── JumpSearchSolver.java
│   │   │   │   └── ExponentialSearchSolver.java
│   │   │   │
│   │   │   ├── sort/
│   │   │   │   ├── MergeSortSolver.java
│   │   │   │   ├── QuickSortSolver.java
│   │   │   │   ├── HeapSortSolver.java
│   │   │   │   └── CountingSortSolver.java
│   │   │   │
│   │   │   ├── graph/
│   │   │   │   ├── BfsSolver.java
│   │   │   │   ├── DfsSolver.java
│   │   │   │   ├── DijkstraSolver.java
│   │   │   │   ├── BellmanFordSolver.java
│   │   │   │   ├── FloydWarshallSolver.java
│   │   │   │   ├── TopoSortSolver.java
│   │   │   │   ├── KruskalMstSolver.java
│   │   │   │   └── PrimMstSolver.java
│   │   │   │
│   │   │   ├── dp/
│   │   │   │   ├── KnapsackSolver.java
│   │   │   │   ├── LcsSolver.java
│   │   │   │   ├── LisSolver.java
│   │   │   │   ├── EditDistanceSolver.java
```

```
│   │   │   │   │   └── CoinChangeSolver.java
│   │   │   │   │
│   │   │   │   ├── backtracking/
│   │   │   │   │   ├── NQueensSolver.java
│   │   │   │   │   ├── SudokuSolver.java
│   │   │   │   │   ├── MazeSolver.java
│   │   │   │   │   └── CombinationSumSolver.java
│   │   │   │   │
│   │   │   │   └── string/
│   │   │   │       ├── KmpSolver.java
│   │   │   │       ├── RabinKarpSolver.java
│   │   │   │       ├── ZAlgorithmSolver.java
│   │   │   │       └── BoyerMooreSolver.java
│   │   │   │
│   │   │   ├── strategy/
│   │   │   │   ├── sorting/
│   │   │   │   │   ├── SortingStrategy.java
│   │   │   │   │   ├── MergeSortStrategy.java
│   │   │   │   │   ├── QuickSortStrategy.java
│   │   │   │   │   └── HeapSortStrategy.java
│   │   │   │   │
│   │   │   │   ├── routing/
│   │   │   │   │   ├── HeuristicStrategy.java
│   │   │   │   │   ├── ManhattanHeuristic.java
│   │   │   │   │   └── EuclideanHeuristic.java
│   │   │   │   │
│   │   │   │   └── compression/
│   │   │   │       ├── CompressionStrategy.java
│   │   │   │       ├── HuffmanCompressionStrategy.java
│   │   │   │       └── LZCompressionStrategy.java
│   │   │   │
│   │   │   ├── factory/
│   │   │   │   ├── AlgorithmFactory.java
│   │   │   │   ├── SearchSolverFactory.java
│   │   │   │   ├── SortSolverFactory.java
│   │   │   │   └── GraphSolverFactory.java
│   │   │   │
│   │   │   ├── ds/                # ✅Data Structures implementations
│   │   │   │   ├── graph/
│   │   │   │   │   ├── Graph.java
│   │   │   │   │   ├── WeightedGraph.java
│   │   │   │   │   ├── GraphNode.java
│   │   │   │   │   ├── GraphEdge.java
│   │   │   │   │   └── AdjacencyListGraph.java
│   │   │   │   │
│   │   │   │   ├── tree/
│   │   │   │   │   ├── TreeNode.java
│   │   │   │   │   ├── BinarySearchTree.java
│   │   │   │   │   ├── AVLTree.java
│   │   │   │   │   ├── SegmentTree.java
```

```
│   │   │   │   └── FenwickTree.java
│   │   │   │
│   │   │   ├── heap/
│   │   │   │   ├── MinHeap.java
│   │   │   │   ├── MaxHeap.java
│   │   │   │   └── PriorityQueueWrapper.java
│   │   │   │
│   │   │   ├── unionfind/
│   │   │   │   └── UnionFind.java
│   │   │   │
│   │   │   └── trie/
│   │   │       └── Trie.java
│   │   │
│   │   ├── cache/
│   │   │   ├── MemoizationCache.java
│   │   │   ├── LruCache.java
│   │   │   └── FrequencyCounterCache.java
│   │   │
│   │   ├── validator/
│   │   │   ├── InputValidator.java
│   │   │   ├── GraphValidator.java
│   │   │   ├── SortValidator.java
│   │   │   └── SecurityValidator.java
│   │   │
│   │   ├── comparator/
│   │   │   ├── SalaryComparator.java
│   │   │   ├── RankComparator.java
│   │   │   └── DistanceComparator.java
│   │   │
│   │   ├── mapper/
│   │   │   ├── AlgorithmMapper.java
│   │   │   └── GraphMapper.java
│   │   │
│   │   ├── exception/
│   │   │   ├── AlgoException.java
│   │   │   ├── InvalidInputException.java
│   │   │   └── GraphNotConnectedException.java
│   │   │
│   │   ├── utils/
│   │   │   ├── SortUtils.java
│   │   │   ├── SearchUtils.java
│   │   │   ├── MathUtils.java
│   │   │   ├── HashUtils.java
│   │   │   ├── EncodingUtils.java
│   │   │   └── TimeComplexityUtils.java
│   │   │
│   │   ├── constants/
│   │   │   ├── AlgoConstants.java
│   │   │   └── ErrorMessages.java
│   │   │
```

```
│   │   └── testdata/
│   │       ├── SampleGraphs.java
│   │       └── SampleInputs.java
│   │
│   └── resources/
│       ├── application.yml
│       ├── logback-spring.xml
│       └── static/
│
└── test/
    └── java/
        └── com/company/algorithmhub/
            ├── solver/
            │   ├── search/
            │   │   └── BinarySearchSolverTest.java
            │   ├── sort/
            │   │   └── QuickSortSolverTest.java
            │   └── dp/
            │       └── KnapsackSolverTest.java
            ├── service/
            │   └── AlgorithmServiceTest.java
            └── controller/
                └── AlgorithmControllerTest.java
```

✅ 2) Where Each Algorithm Class Belongs (Simple Mapping)

Class Category	Package	What goes here
REST API endpoints	controller/	SortController, GraphController
Input/Output models	dto/request, dto/response	SortRequest, GraphPathResponse
Business layer (algo + business)	service/	FraudDetectionService, RecommendationService
Multi-step pipelines	engine/	SearchEngine, IndexingEngine
Pure algorithm execution	solver/	DijkstraSolver, KnapsackSolver
Algorithms swappable	strategy/	SortingStrategy, CompressionStrategy
Algo selection object	factory/	AlgorithmFactory, SortSolverFactory
Data structures	ds/	Trie, UnionFind, SegmentTree
Memoization/caching	cache/	MemoizationCache, LruCache

Validations	validator/	InputValidator, GraphValidator
Custom ordering	comparator/	DistanceComparator
Shared helpers	utils/	MathUtils, HashUtils
App configs	config/	SwaggerConfig, CacheConfig
Exceptions/errors	exception/	InvalidInputException

✅ 3) Clean Naming Rules (Industry Standard)

✅ Utility classes

✅ Contains only static methods

```
public final class SortUtils {
   private SortUtils() {}

   public static int[] mergeSort(int[] arr) { ... }
}
```

✅ Solver classes

✅ Clean algorithm execution class

```
public class DijkstraSolver {
   public Map<Integer, Integer> shortestPath(WeightedGraph graph, int source) { ... }
}
```

✅ Engine classes

✅ Multiple steps / pipeline

```
public class RecommendationEngine {
   public List<ProductDTO> recommend(UserDTO user) {
      // load history -> compute similarity -> rank -> filter
   }
}
```

✅ Strategy classes

✅ interchangeable algorithms

```
public interface SortingStrategy {
   int[] sort(int[] arr);
}
```

✅ 4) Most Important Rule (Interview Point)

✅ **Algorithm classes should NEVER be inside Controller.**

Controller should only:

- take request
- validate request
- call service
- return response

If you want, I can also generate:

✅ **Complete Spring Boot project skeleton (zip-style)**
✅ with **one API for every algorithm category** + unit tests + Swagger UI